

Version Control & Git Basics

Foundation Course @ H-BRS

Jaqueline Machida

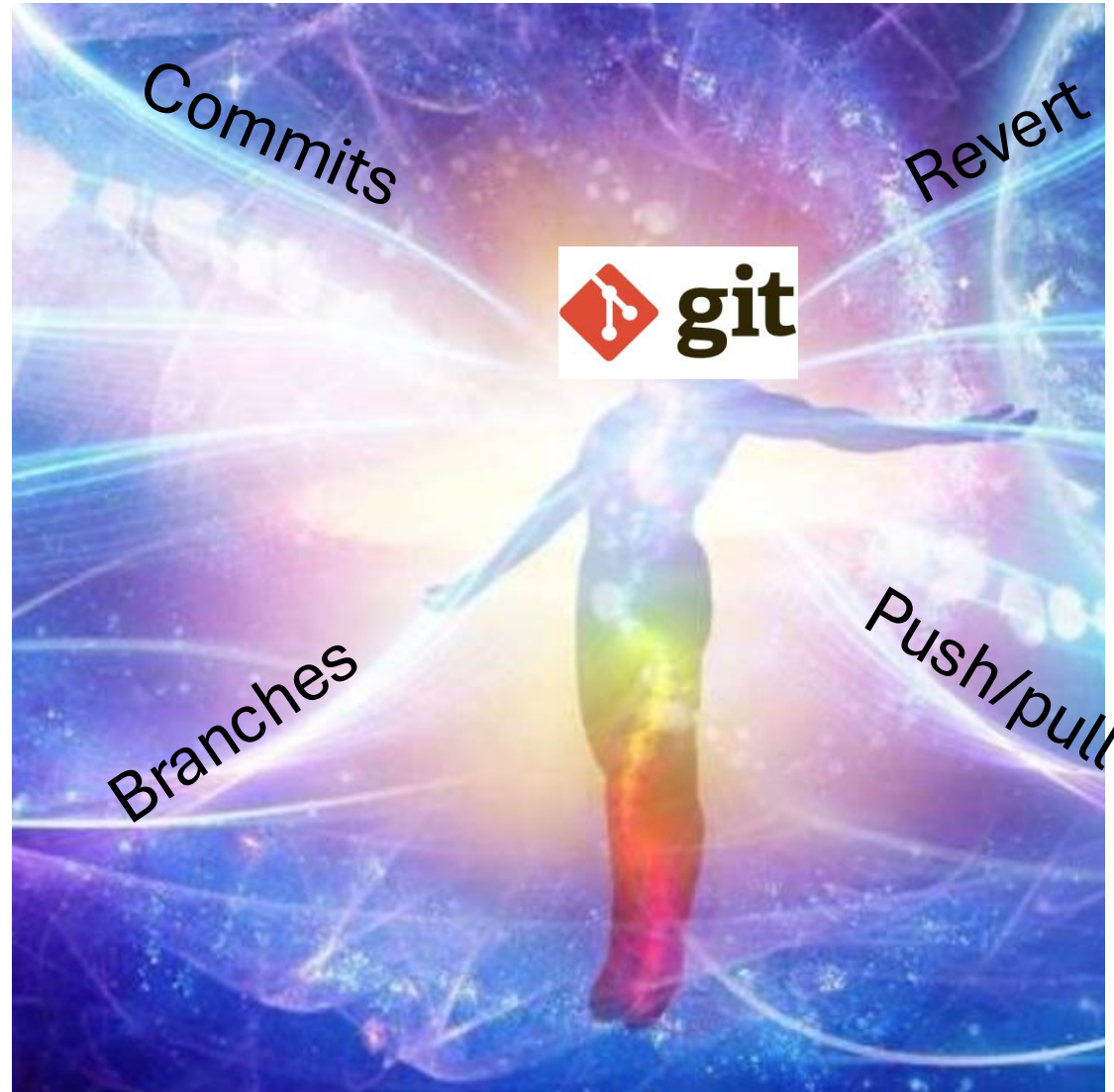
September 2026

Before we start... who are the new MAS students?

Introduction to myself

What all those projects have in common?

What all those projects have in common?



Version Control



Track of changes

Who and what

Revert changes

Collaboration

Git



Version Control System (tool)

Most widely used

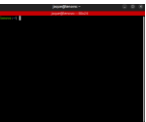
Homework?



Getting started (homework)

- Git installed in your machine ([link](#))
- Your SSH key created and added to the ssh-agent ([link](#))
- Your SSH key added to your GitHub account ([link](#))
- For the previous step, create a GitHub account in case you don't have one yet ([link](#))

I didn't



HTTPS vs SSH

To collaborate on GitHub (cloning/pulling/pushing code), you can use two methods:

- HTTPS (username and password)
- SSH (using a pair of cryptographic keys)

Global/local identity

Global identity: applies to every repository on your machine

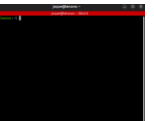
```
git config --global user.name "John Doe"
```

```
git config --global user.email johndoe@example.com
```

Global/local identity

Local identity: applies only to the one you're currently in

```
cd my-project  
git config user.name "John Doe (Work)"  
git config user.email john.doe@company.com
```



Concepts

Commit

Branch

Merge

Conflict

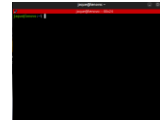
Origin

Staging

Repository

Concepts

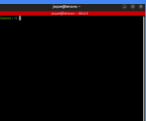
working directory → staging area → repository (commits)



Local vs remote repositories

Origin: remote URL

GitHub { Stores repositories in the cloud
GitLab, BitBucket...
Resources during your studies ([link](#))



Cloning a repository

```
git clone git@github.com:JaquelineSayuri/git_introduction.git
```

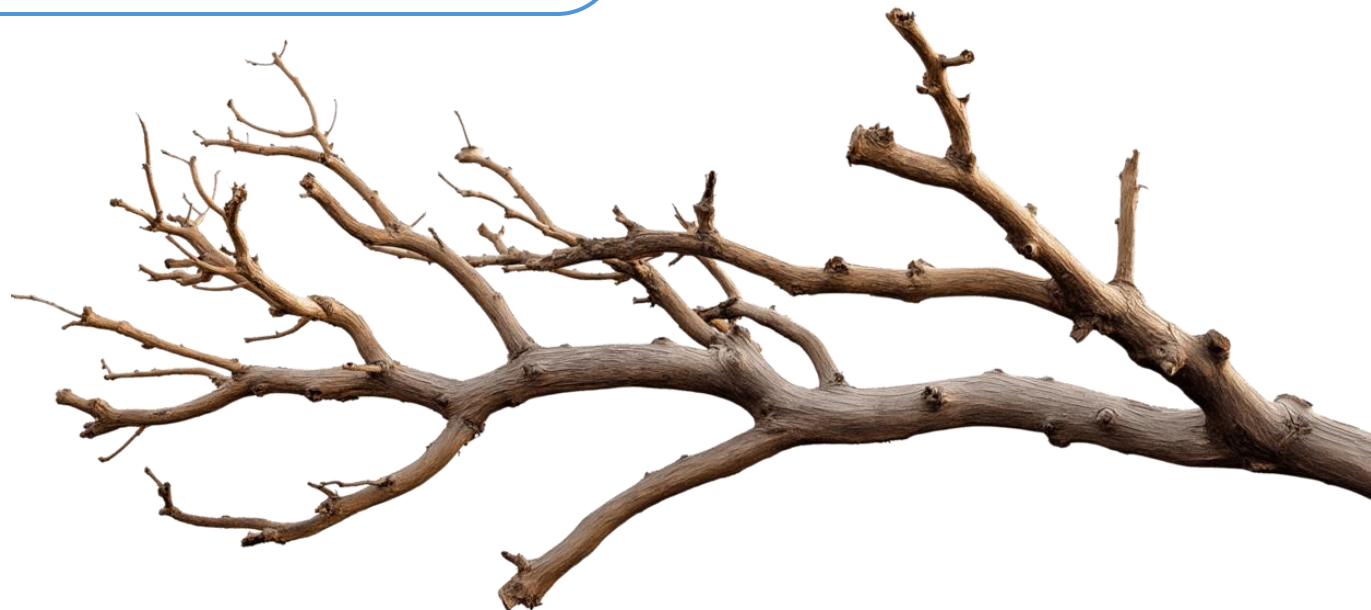
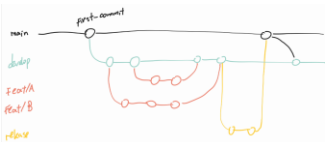


Branches

```
git branch feature-login    # create a new branch
```

```
git checkout feature-login    # switch to it
```

```
git checkout -b feature-login # do both in one step
```



Main branch

main vs. master

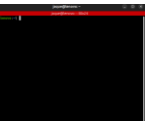
```
git config --global init.defaultBranch <name>
```

```
git branch -m <name>
```

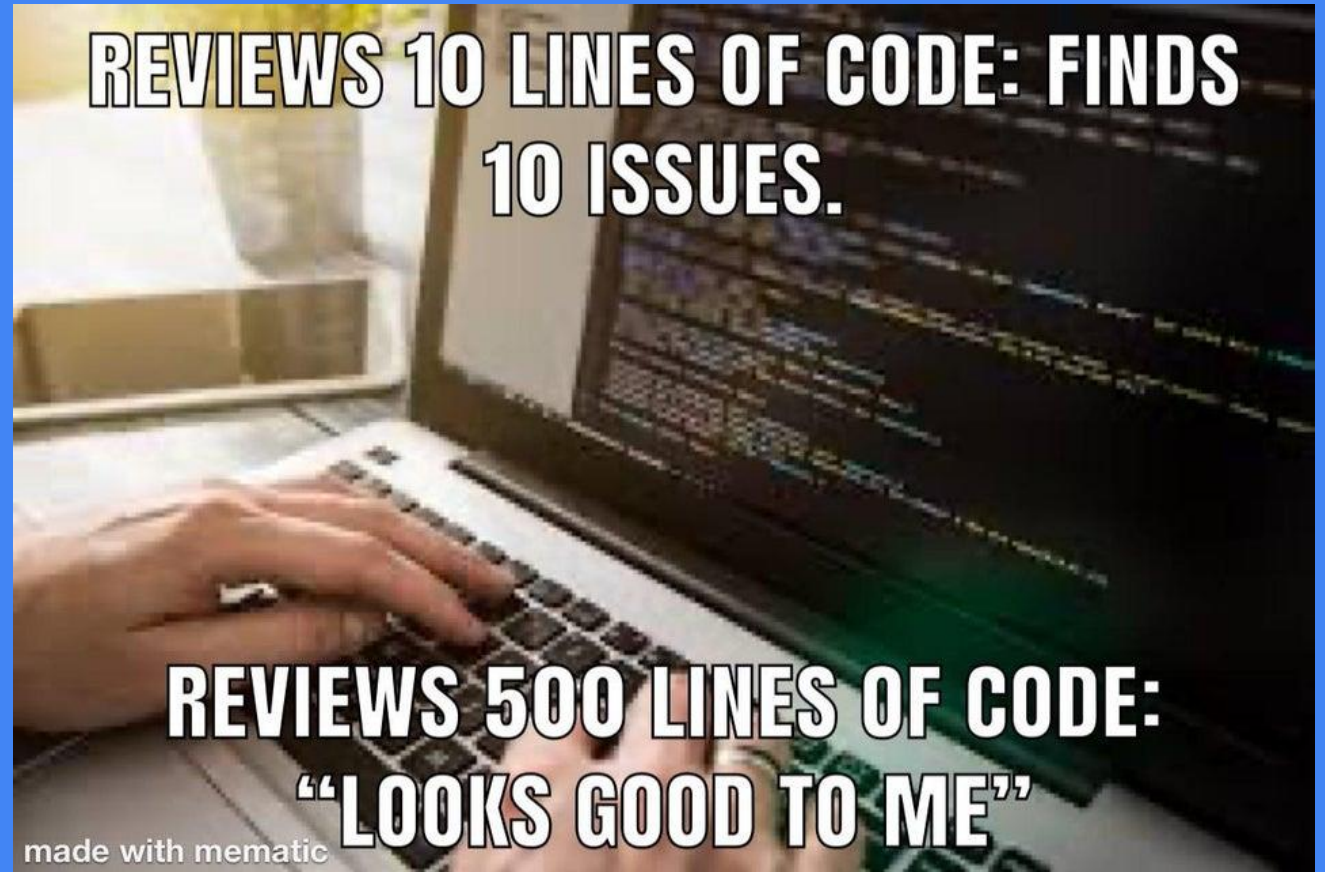
Push/pull

```
git push origin [branch-name]
```

```
git pull origin [branch-name]
```



Pull request

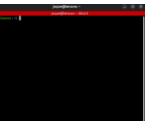


Stash

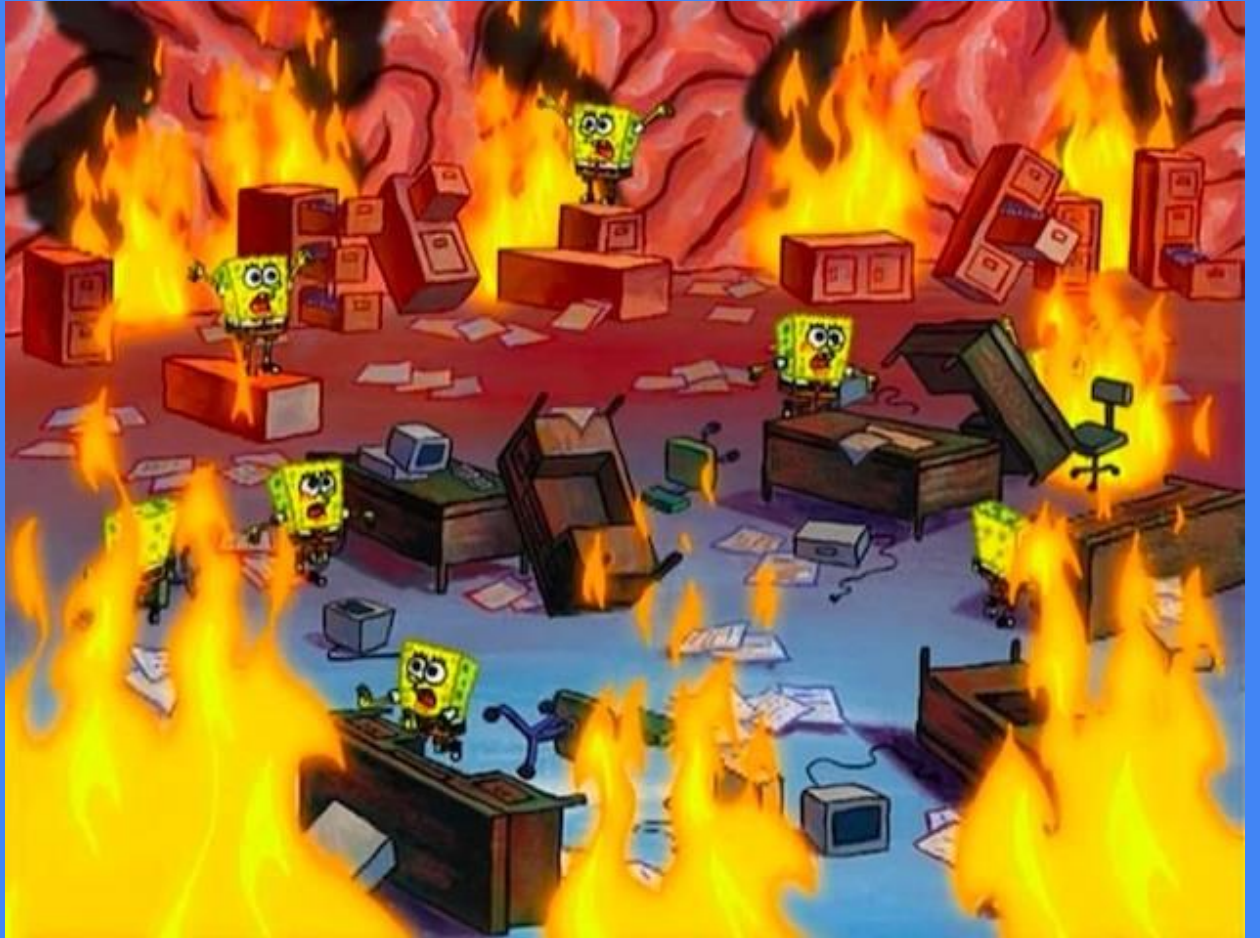
git stash

git stash list

git stash apply



Group work



1. Divide into groups
2. One person in each group should create a new GitHub repository
3. Invite your team members as collaborators
4. Let everyone clone the repository
5. Everyone should checkout a new branch with their name
6. Create a file, write something nice about yourself!
7. Push it
8. Create a pull request
9. Merge all pull requests

Wait a minute



Seeing History & Going Back

Viewing history: `git log`

Seeing what changed:

```
git diff
```

```
git diff HEAD~1 HEAD
```

```
git diff a1b2c3d e4f5g6h
```

Seeing History & Going Back

Three ways to "go back":

1. `git checkout <commit>`: look at the past, temporarily

```
git checkout a1b2c3d
```

```
git checkout main
```

Seeing History & Going Back

Three ways to "go back":

2. git reset: rewrite history by deleting commits

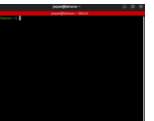
```
git reset --hard HEAD~1
```

Seeing History & Going Back

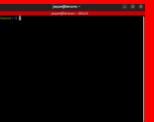
Three ways to "go back":

3. `git revert <commit>`: undo safely by adding a new commit

```
git revert e4f5g6h
```

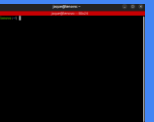


Conflicts



More concepts

.gitignore



Commit message template

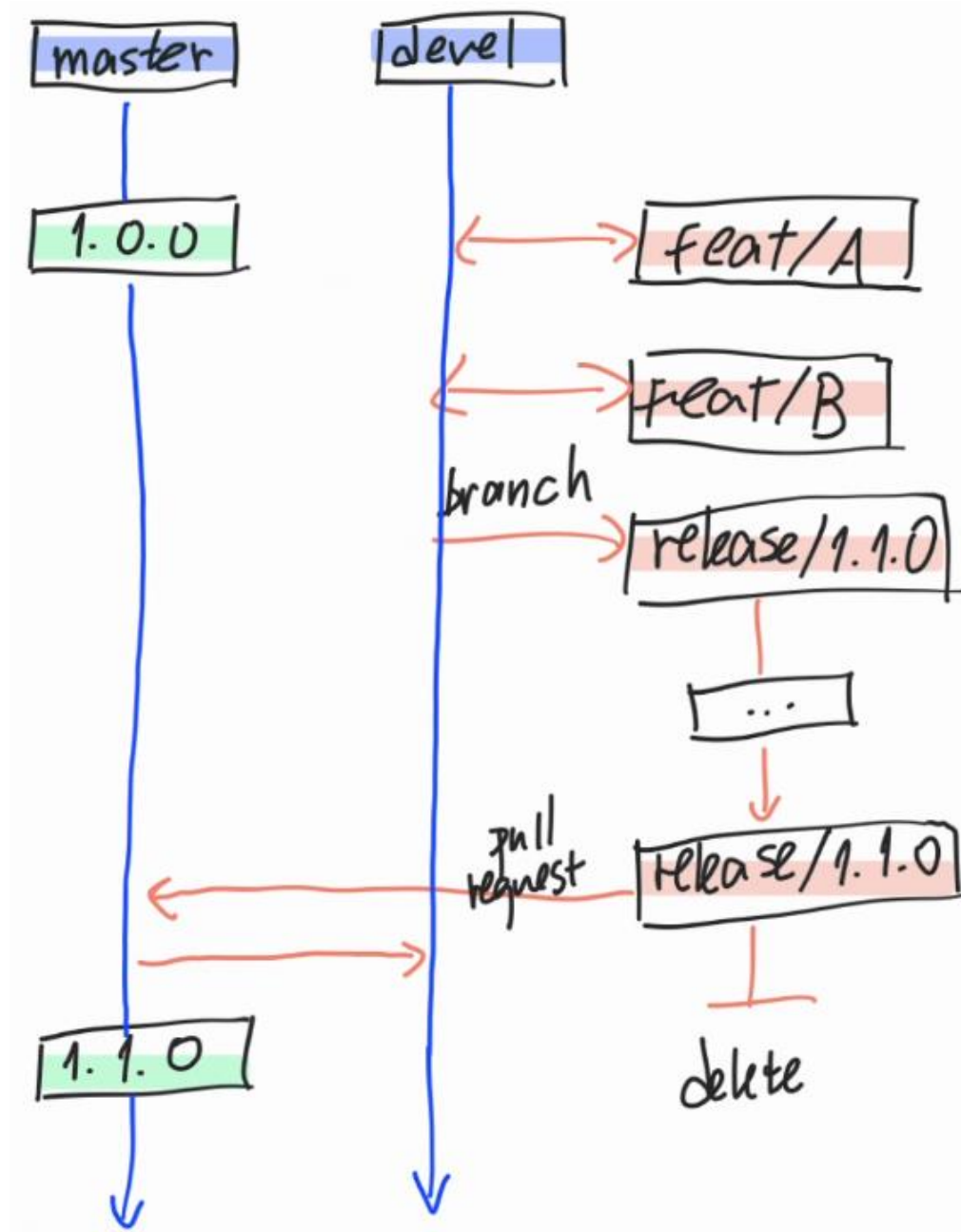
<type>: <title>

<body>

<footer>

([link](#))

Git Flow



Even more concepts

Just kidding. References

<https://git-scm.com/install/linux>

<https://git-scm.dev/doc>

<https://docs.github.com>

<https://github.com>

Previous Foundation Course slides/materials

Claude AI

Voices in my head

Years of ~~suffering~~ experience

Thank you!
Let's connect

